



# LABORATORIO DI RETI INFORMATICHE

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA  
A.A. 2025/2026

Francesca Righetti  
[francesca.righetti@unipi.it](mailto:francesca.righetti@unipi.it)



# ESERCITAZIONE 8

Programmazione con i socket II - Esercizi

# Programma

- Esercizi di programmazione distribuita

# Esercizio 3 – Echo Server 2.0

- Rendere multi-processo il server dell'esercizio «Echo Server» della scorsa volta usando la primitiva `fork()`
- Rimuovere il limite dei 20 byte: il client invia la dimensione esatta della stringa e il server legge la dimensione esatta di byte.
  - Come fa il server a sapere in anticipo quanti byte leggere?

# Esercizio 3 – Echo Server 2.0

- Rimuovere il limite dei 20 byte: il client invia la dimensione esatta della stringa e il server legge la dimensione esatta di byte.
  - Come fa il server a sapere in anticipo quanti byte leggere?

```
... //client
char buffer[BUFFER_SIZE] = {0};
int net_message_length;

...
int message_length = strlen(buffer); // Invio al server la quantita di dati
net_message_length = htonl(message_length); // Converto in formato network
ret = send(sd, &net_message_length, sizeof(net_message_length), 0); // Invio il numero di dati
ret = send(sd, buffer, message_length, 0); // Invio I dati
```

```
... //server
ret = recv(new_sd, &message_length, sizeof(message_length), 0); // Ricevo la quantita di dati
...
message_length = ntohl(message_length); // Rinverto in formato host
ret = recv(new_sd, buffer, message_length, 0); // Ricevo I dati
```

# Socket non bloccante

- Serve per manipolare un descrittore di file o di socket
  - fd è il descrittore
  - cmd è il comando da passare
    - F\_GETFL: Ottieni i flag di stato
    - F\_SETFL: Imposta i flag di stato
  - In base al comando può esserci un terzo argomento
  - Il valore restituito dipende dal comando passato, -1 e errno su errore

```
#include <unistd.h>
#include <fcntl.h>

int fcntl(int fd, int cmd, ...);
```

# Socket non bloccante

- I flag vengono passati tutti insieme, quindi bisogna salvare quelli impostati e riscriverli insieme ai nuovi

```
int sd, ret;

sd = socket(...);

/* Ottieni i flag attuali */
ret = fcntl(sd, F_GETFL, NULL);

/* Setta il flag non bloccante, mantenendo gli altri */
fcntl(sd, F_SETFL, ret | O_NONBLOCK);
```

# Stampare data e ora

```
time_t rawtime;
struct tm * timeinfo;
/* Ottieni l'ora in formato POSIX */
time(&rawtime);
timeinfo = localtime (&rawtime);
/* La struttura timeinfo non va deallocata (è allocata staticamente dal
sistema operativo, viene sovrascritta ad ogni invocazione */

/* Stampa l'ora */
// ctime() trasforma l'ora in stringa
printf("%s\n", ctime(&rawtime));
/* Formattare data/ora in stringa secondo un formato specificato */
strftime(buffer, BUFFER_SIZE, "%Y-%m-%d %H:%M:%S", timeinfo);
```

- %A: Nome completo del giorno della settimana (es. "Wednesday").
- %d: Giorno del mese, con due cifre (es. "25").
- %B: Nome completo del mese (es. "September").
- %Y: Anno con quattro cifre (es. "2024").
- %H: Ora con due cifre nel formato 24 ore (es. "15").
- %M: Minuti con due cifre (es. "30").
- %S: Secondi con due cifre (es. "45").

- %a: Nome abbreviato del giorno della settimana (es. "Mon", "Tue").
- %b: Nome abbreviato del mese (es. "Jan", "Feb").
- %I: Ora con due cifre nel formato 12 ore (es. "03").
- %p: AM/PM (usato con il formato 12 ore).
- %m: Mese (01-12).



# Struct tm

```
struct tm{
    int    tm_sec    // Seconds [0,60].
    int    tm_min    // Minutes [0,59].
    int    tm_hour   // Hour [0,23].
    int    tm_mday   // Day of month [1,31].
    int    tm_mon    // Month of year [0,11].
    int    tm_year   // Years since 1900.
    int    tm_wday   // Day of week [0,6] (Sunday =0).
    int    tm_yday   // Day of year [0,365].
    int    tm_isdst  // Daylight Savings flag.
};
```

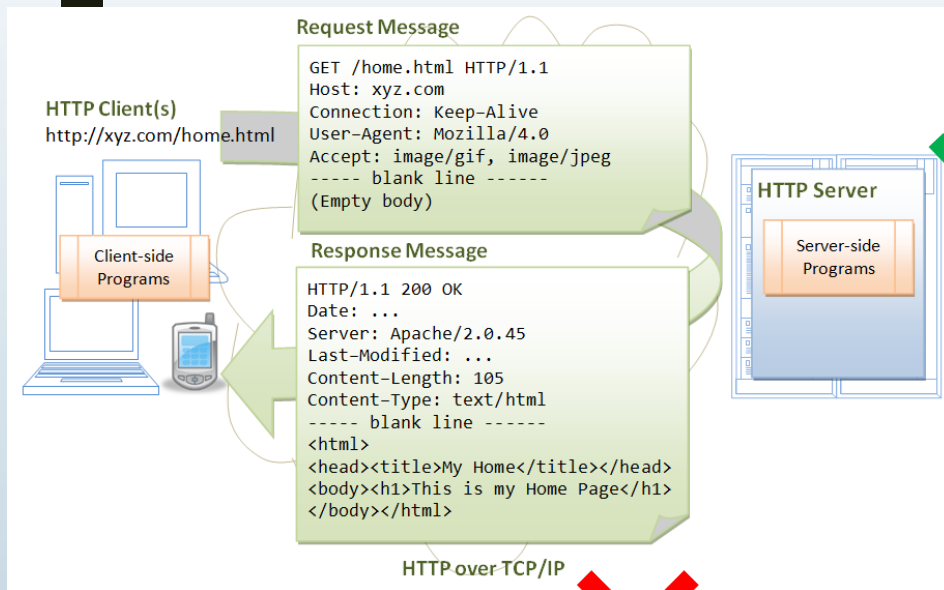
# Esercizio 4 – Time Server

- Implementare un semplice server UDP che periodicamente (10s) invia l'ora ai client che si registrano
  - Il server periodicamente controlla se c'è una richiesta di registrazione, se c'è registra il client, poi invia a tutti i client registrati un pacchetto UDP contenente la data e l'ora
- Il client invia la richiesta e poi aspetta il dato, stampandola ogni volta che arriva
- Non usare fork() o select()

# Text Vs Binary Protocols

- Molti protocolli a livello applicativo inviano messaggi di testo (text protocols) invece di inviare direttamente i dati delle strutture dati (binary protocols)
- I dati da inviare sono tradotti in stringhe e inviati nel socket come testo
- I campi del messaggio vengono descritti usando delle parole chiave
- Il protocollo definisce una lunghezza massima dei messaggi

# Text Protocols



- HyperText Transfer Protocol **HTTP**
- Simple Mail Transfer Protocol **SMTP**
- File Transfer Protocol **FTP**

✓ **Leggibilità:** Si può leggere facilmente ciò che viene trasmesso.

- **Facilità di implementazione:** È più semplice creare e interpretare un protocollo testuale, dato che il formato è intuitivo.
- **Compatibilità umana:** il testo è leggibile, strumenti come telnet, curl o netcat possono essere utilizzati per inviare o ricevere messaggi e fare test.
- **Estendibilità:** Puoi facilmente aggiungere nuovi campi o comandi senza rompere la compatibilità, poiché puoi ignorare parti del testo che non riconosci.

- ✗
- **Efficienza:** I dati trasmessi sono generalmente più grandi rispetto a un protocollo binario, perché numeri o dati binari devono essere convertiti in stringhe. Ad esempio, inviare un intero a 32 bit come stringa (ad es. "1234567890") richiede più byte rispetto alla sua rappresentazione binaria.
  - **Parsing lento:** Il parsing del testo può essere più lento rispetto a un protocollo binario, poiché è necessario interpretare e convertire il testo nei tipi di dati corretti.
  - **Ambiguità:** In alcuni casi, la gestione di formati o delimitatori può portare a errori se non è ben definito o implementato.

# Text Protocol: convertire struttura in stringa

- Nel caso della nostra struttura questa potrebbe essere convertita in testo:

```
struct temp{  
    int a;  
    char b;  
};  
struct temp t;  
sprintf(buffer,"%d %c",t.a, t.b);
```

# Text Protocol: convertire struttura in stringa

- Al ricevente convertire il messaggio:

```
struct temp t;  
sscanf(buffer, "%d %c", &t.a, &t.b);
```

# Binary Protocol

- Utilizza dati codificati direttamente nel formato binario nativo, non leggibile dall'uomo. Invece di inviare i dati come testo, vengono inviati nella loro rappresentazione binaria (come numeri, strutture, ecc.).
- Si definiscono dei messaggi con una struttura prefissata, composti da campi che rappresentano l'informazione da scambiare
  - Se devo inviare "12345", viene inviato come un intero a 32 bit con il valore **12345** in formato binario, che occupa 4 byte (anziché 5 caratteri di testo più eventuali spazi o delimitatori nel caso si invii con un text protocol).
- Ciascun campo ha una lunghezza e un tipo associate
- Alcuni protocolli di tipo binario potrebbero contenere campi a lunghezza variabile, in quel caso comunque il protocollo definisce una lunghezza massima dei messaggi

# Inviare una struttura su un socket

38 I am trying to pass whole structure from client to server or vice-versa. Let us assume my structure as follows

```
37 struct temp {  
    int a;  
    char b;  
}
```

I am using **sendto** and sending the address of the structure variable and receiving it on the other side using the **recvfrom** function. But I am not able to get the original data sent on the receiving end. In sendto function I am saving the received data into variable of type struct temp.

```
n = sendto(sock, &pkt, sizeof(struct temp), 0, &server, length);  
n = recvfrom(sock, &pkt, sizeof(struct temp), 0, (struct sockaddr *)&from, &fromlen);
```

Where pkt is the variable of type struct temp.

Eventhough I am receiving 8bytes of data but if I try to print it is simply showing garbage values. Any help for a fix on it ?

59 This is a very bad idea. Binary data should always be sent in a way that:

- Handles different **endianness**
- Handles different **padding**
- Handles differences in the **byte-sizes of intrinsic types**

Don't ever write a whole struct in a binary way, not to a file, not to a socket.

Always write each field separately, and read them the same way.

You need to have functions like

```
unsigned char * serialize_int(unsigned char *buffer, int value)  
{  
    /* Write big-endian int value into buffer; assumes 32-bit int and 8-bit char. */  
    buffer[0] = value >> 24;  
    buffer[1] = value >> 16;  
    buffer[2] = value >> 8;  
    buffer[3] = value;  
    return buffer + 4;  
}  
  
unsigned char * serialize_char(unsigned char *buffer, char value)  
{  
    buffer[0] = value;  
    return buffer + 1;  
}  
  
unsigned char * serialize_temp(unsigned char *buffer, struct temp *value)  
{  
    buffer = serialize_int(buffer, value->a);  
    buffer = serialize_char(buffer, value->b);  
    return buffer;  
}
```

Altrimenti usare la  
htonl()



# Binary Protocol: Invio

- La struttura deve essere progettata già con dei tipi che possano essere trasferiti:

```
struct temp{
    uint32_t a;
    uint8_t b;
};
struct temp t;
...
t.a = htonl(t.a); // Convertire il formato prima dell'invio
ret = send(new_sd, (void*)&t.a, sizeof(uint32_t), 0);
t.b = htonl(t.b);
ret = send(new_sd, (void*)&t.b, sizeof(uint8_t), 0);
```

```
struct temp t;
ret = recv(new_sd, (void *)&t.a,
sizeof(uint32_t), 0);
if (ret < sizeof(uint32_t)) { // Gestione errore}
t.a = ntohl(t.a); // Convertire in host order il
campo 'a'
ret = recv(new_sd, (void *)&t.b, sizeof(uint8_t),
0);
if (ret < sizeof(uint8_t)) { // Gestione errore}...
```

- uint32\_t è un tipo di dato con una dimensione fissa di **32 bit** (4 byte) su tutte le piattaforme.
- Questo è fondamentale per i protocolli binari, dove è necessario sapere esattamente quanti byte vengono trasmessi e ricevuti, indipendentemente dall'architettura o dal sistema operativo.
  - int, ha una dimensione che **può variare** tra diverse architetture e compilatori:
    - su sistemi a 32 bit, un int è di 4 byte.
    - su sistemi a 64 bit, un int può essere di 4 byte, ma non è garantito.
- Questa variabilità rende l'uso di int poco sicuro nei protocolli binari, perché un valore che viene inviato come int da un sistema potrebbe essere interpretato in modo errato da un altro sistema che utilizza una dimensione diversa per int.

# Binary Protocol: Invio

- Nei protocolli binari, i dati sono inviati nella loro forma binaria nativa
- Tuttavia, la rappresentazione binaria dei dati può variare tra diverse piattaforme, quindi è necessario gestire l'**endianness** e la **serializzazione/deserializzazione**
  - Per garantire la corretta interpretazione dei dati tra sistemi con endianness diverso, si utilizzano funzioni come `htonl()` (Host To Network Long) e `ntohl()` (Network To Host Long) per convertire tra **host byte order** e **network byte order**
  - La serializzazione è il processo di trasformare una struttura complessa in una sequenza di byte, mentre la deserializzazione fa il contrario, trasformando una sequenza di byte in una struttura dati

# Binary Protocol: Invio



- **Efficienza:** Un protocollo binario utilizza meno spazio rispetto a un protocollo testuale. Dati come numeri, floating point o strutture di dati possono essere inviati nella loro rappresentazione nativa.
- **Velocità:** Il parsing di un protocollo binario è generalmente più veloce, poiché non è necessario convertire da stringhe di testo a tipi di dati nativi.
- **Precisione:** Non ci sono problemi di formattazione o ambiguità legata alla rappresentazione del testo. Le informazioni vengono trasmesse esattamente come sono in memoria.



- **Difficoltà di debug:** Poiché i dati sono trasmessi in formato non leggibile dall'uomo, il debug manuale diventa più difficile. È necessario uno strumento specializzato per visualizzare e interpretare i dati binari (come un analizzatore di pacchetti tipo Wireshark).
- **Estendibilità:** Estendere un protocollo binario può essere più difficile, perché i messaggi devono rispettare una struttura fissa e rigorosa. Aggiungere nuovi campi potrebbe richiedere modifiche ai byte già esistenti, rompendo la compatibilità.
- **Compatibilità tra sistemi:** I protocolli binari devono affrontare problemi di byte order (endianness) tra sistemi diversi (big-endian vs little-endian), e richiedono funzioni come `htonl()` e `ntohl()` per la conversione.

# Esercizio 5 – Time Server TCP

- Modificare il time server (Esercizio 4) in modo che utilizzi il protocollo TCP
- I client rimangono connessi e periodicamente inviano la richiesta e poi aspettano il dato, stampandola ogni volta che arriva
- Inviare attraverso il socket solamente l'ora in hh:mm:ss
- Usare la select() per gestire richieste multiple da parte dei client